



SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA3712

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Code of Conduct:

*I **JEEVA K (192421112)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1: Testing as an Engineering Activity

Question: A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer

Random testing depends on individual guesswork, so results vary from person to person. Treating testing as an engineering activity means it is planned, designed, executed and measured using defined methods, just like any other engineering discipline. This replaces chance with a repeatable process.

Key Points

- Define test objectives and scope from the requirements and a risk analysis.
- Derive test cases with proven design techniques such as equivalence partitioning, boundary value analysis, decision tables and state transition testing.
- Maintain a requirements traceability matrix so that every requirement has at least one test case.
- Specify test data, entry/exit criteria and expected results before execution begins.
- Record results, log defects and track metrics such as defect density, test coverage and defect detection efficiency.

Example

For a sign-up form, the redesigned approach derives cases for a valid e-mail, a missing '@', an empty field and a maximum-length value, each with a predefined expected result. Any defect found can be reproduced and traced to its requirement.

Conclusion

Structured test design gives wider coverage, repeatable results and measurable progress, which directly improves quality and defect detection.

Question 2: Well-Defined Testing Process in Banking Systems

Question: *A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.*

Answer

A well-defined testing process is a documented sequence of activities: test planning, test design, test execution, defect reporting and test closure. Banking systems are real-time and mission-critical, so a failure can mean financial loss and loss of customer trust. A standard process makes testing consistent and auditable.

Key Points

- Consistency: every release is tested with the same discipline, so no critical area is skipped.
- Early detection: planned reviews and test design expose defects before they reach production.
- Risk control: high-risk functions such as transfers, interest calculation and balance updates receive priority.
- Traceability and audit: documented evidence supports regulatory and compliance checks.
- Measurable improvement: metrics from each cycle show where the process needs strengthening.

Example

In a funds-transfer module, the defined process requires test cases for insufficient balance, duplicate submission, simultaneous transfers and rollback on failure. A double-debit defect found during execution is fixed before release instead of being discovered by customers.

Conclusion

A defined process turns testing from an occasional activity into a controlled quality gate, reducing failures in real-time banking systems.

Question 3: Structured Testing within the SDLC

Question: During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer

Testing should not be a single activity at the end of development. It should run in parallel with every SDLC phase, as shown by the V-model, where each development phase is paired with a corresponding test level.

Key Points

- Requirements phase: prepare the acceptance test plan and review requirements for testability.
- System design phase: prepare system test cases covering end-to-end behaviour.
- High-level design phase: plan integration tests for module interfaces.
- Low-level design and coding phase: write and run unit tests; perform code reviews.
- Each phase has entry and exit criteria, so work cannot move forward until quality conditions are met.

Example

In an e-commerce platform, integration tests for the cart-to-payment interface are planned during design. A mismatch in the order-amount data format is therefore found during integration testing instead of appearing as a system-level failure.

Conclusion

Process discipline (plans, reviews, entry/exit criteria and regression) removes gaps between phases, so defects are caught at the level where they are cheapest to fix.

Question 4: Error, Defect and Failure

Question: A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer

The three terms describe different stages of the same problem. An error is a human mistake made by a developer or analyst. A defect (fault or bug) is the flaw in the software that results from that mistake. A failure is the observable deviation from expected behaviour that occurs when the defective code is executed.

Key Points

- Error (mistake) leads to a defect (fault in the code or design).
- A defect leads to a failure only when the faulty code is executed under the right conditions.
- The login mismatch between expected and actual output is a failure, because it is the visible symptom.
- The incorrect password-comparison logic behind it is the defect.
- The developer's wrong assumption or coding slip is the error.

Example

A developer mistakenly writes a case-insensitive password comparison (error). This produces a flawed condition in the code (defect). When a user logs in with 'ABC123' instead of 'abc123' and is still allowed in, the system exhibits a failure.

Conclusion

The issue should be classified as a failure, traced to a defect, and caused by an error. Understanding the chain helps teams fix the root cause rather than the symptom.

Question 5: Early Testing Principle

Question: A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer

The principle that addresses late defect detection is Early Testing: testing activities should begin as early as possible in the lifecycle, starting with requirements and design, not only after code is complete. The cost of correcting a defect rises sharply with each phase in which it remains undetected.

Key Points

- Review requirements and design documents to find defects before any code exists.
- Begin writing test cases in parallel with requirement analysis.
- Run unit tests and static analysis as soon as code is written.
- Use continuous integration so that each change is verified quickly.
- Fixing a defect at the requirement stage typically costs a fraction of fixing it in production.

Example

A team reviews a requirement stating 'discount applies to large orders'. The tester asks what 'large' means and the ambiguity is resolved immediately. Without the review, the defect would surface in system testing and require rework in design, code and test cases.

Conclusion

Applying early testing reduces rework and cost, shortens schedules and increases the number of defects found before release.

Question 6: Tester's Contribution during Requirement Analysis

Question: During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer

A tester contributes during the requirement phase by examining requirements from a verification point of view. A tester naturally asks how each statement will be checked, which exposes ambiguity, omissions and contradictions before design begins.

Key Points

- Participate in requirement reviews, walkthroughs and inspections.
- Check each requirement for clarity, completeness, consistency and testability.
- Raise queries on ambiguous or missing items and obtain written clarification.
- Define acceptance criteria and begin drafting test scenarios.
- Create a traceability matrix linking each requirement to its planned tests.

Example

A healthcare requirement says 'the system shall calculate medicine dosage for patients'. The tester asks which age groups, weight ranges and units apply. The answers lead to precise rules, preventing a dosage defect that could affect patient safety.

Conclusion

Early involvement is important because requirement defects are cheapest to fix at this stage, and clear requirements lead to accurate design, code and test cases.

Question 7: Origin of Defects in the Software Lifecycle

Question: *A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.*

Answer

Defects can enter at any lifecycle phase, but unclear requirements are one of the most common and most expensive sources. An ambiguous requirement is interpreted differently by analysts, developers and testers, and the misunderstanding carries forward into design, code and test cases.

Key Points

- Requirement phase: ambiguity, incompleteness and conflicting statements.
- Design phase: wrong logic or data models based on misread requirements.
- Coding phase: implementation of an incorrect assumption, plus ordinary coding mistakes.
- Testing phase: test cases built on the same misunderstanding fail to reveal the defect.
- Result: the defect reaches production and is discovered as incorrect output.

Example

A payroll requirement states 'overtime is paid at a higher rate' without specifying the rate or the hours threshold. The developer assumes 1.5 times after 40 hours, while the business intended 2 times after 45 hours, so salaries are calculated incorrectly.

Conclusion

Requirement clarity can be achieved through precise, measurable statements, a shared glossary, worked examples, prototypes, formal reviews and acceptance criteria. These prevent misinterpretation at the source.

Question 8: Boundary Defect Classification and Boundary Value Analysis

Question: *A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.*

Answer

A crash that occurs when boundary values are entered is a functional defect of the input-validation / boundary-handling type. It is also a high-severity defect because it terminates the application. It arises from incorrect handling of the limits of an input range.

Key Points

- Classification: functional defect in input validation (boundary condition), coding/logic origin.
- Justification: the system does not behave as specified at the minimum or maximum limit, and the failure is a crash, so severity is high.

- Boundary Value Analysis (BVA) tests values at and around each limit: minimum-1, minimum, minimum+1, maximum-1, maximum and maximum+1.
- Most boundary defects come from off-by-one errors such as using '<' instead of '<='.
- BVA catches these errors systematically with a small number of test cases.

Example

An age field accepts 18 to 60. BVA would test 17, 18, 19, 59, 60 and 61. If the code fails at 60 or 61, the defect is revealed during testing and not by a user.

Conclusion

Applying BVA during test design would have exposed the crash early and improved coverage of the input domain.

Question 9: Defect Repository and Test Case Design

Question: *A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.*

Answer

A defect repository is a central store of all defects found, with details such as module, severity, phase detected, root cause and status. When analysed, it shows what kinds of defects occur and where, and this knowledge can be used to design better test cases.

Key Points

- Analyse defect patterns to find the most frequent defect types and the most defect-prone modules.
- Design additional test cases that target those recurring defect types.
- Convert every fixed defect into a regression test so it cannot return unnoticed.
- Build checklists for reviews and error guessing from past defects.
- Use trends and metrics to decide where to focus testing effort in the next release.

Example

If the repository shows that many past defects involve empty or null values in the order module, the team adds null and empty-input test cases to every similar form. Repeated defects across releases then drop.

Conclusion

A defect repository acts as organisational memory. It improves test design, avoids repeated mistakes and steadily raises software quality.

Question 10: Functional Defect in an ATM Withdrawal

Question: *An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.*

Answer

A functional defect occurs when the software does not perform a function as stated in the requirements. An ATM that fails during withdrawal because of wrong transaction logic is a functional defect, since the core withdrawal feature does not behave as specified.

Key Points

- Description: the withdrawal logic mishandles the sequence of validating the balance, dispensing cash and updating the account.
- Typical symptoms: the balance is debited but no cash is dispensed, or cash is dispensed without a debit.
- Prevention at requirement and design stage: review the transaction flow and model it as a state diagram.
- Prevention at test stage: use decision tables and state transition testing to cover all combinations, including insufficient funds and dispenser failure.
- Additional controls: code reviews, unit tests for the transaction logic and regression testing after every change.

Example

A test case that simulates a cash-dispenser fault after the account has been debited would verify that the transaction is rolled back correctly. Such a scenario, if included, would have revealed the defect before release.

Conclusion

Thorough functional testing of transaction logic, backed by design reviews, would have detected this defect and prevented the field failure.

Question 11: Defect Clustering and Testing Priorities

Question: A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Answer

Defect clustering is the testing principle stating that a small number of modules contain most of the defects, often described by the Pareto rule (roughly 80% of defects in 20% of modules). The testing team can use this to decide where to concentrate its limited effort.

Key Points

- Collect defect data per module and calculate defect density.
- Rank modules by defect count and business risk to set testing priorities.
- Allocate more testers, time and test cases to the high-defect modules.
- Apply deeper techniques there: code reviews, stronger regression and additional negative tests.
- Continue to run lighter tests on the remaining modules so that new problem areas are noticed.

Example

In a large retail application, the payment and checkout modules account for most of the defects. The team schedules more test cycles and reviews for these modules than for static pages such as 'About Us'.

Conclusion

Focusing on defect-prone areas gives the greatest number of defects found per unit of effort, making testing more efficient and cost-effective.

Question 12: Defect Classification: Performance Defect

Question: A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Answer

Slow response under heavy user load is a non-functional defect of the performance (load/scalability) category. The application may give correct results, but it does not meet response-time and capacity expectations.

Key Points

- By type: non-functional defect, subtype performance.
- By severity: usually high or critical, as it affects many users at peak times.
- By origin: commonly design or implementation, such as inefficient queries, lack of caching or poor resource handling.
- Classification routes the defect to the right owner (for example the database or architecture team).
- It also supports prioritisation, trend analysis and root-cause analysis across releases.

Example

Load testing with a tool such as JMeter shows response time rising sharply beyond 1,000 concurrent users. The defect is classified as performance/high severity, assigned to the back-end team, and resolved by adding database indexes and caching.

Conclusion

Proper classification speeds up resolution because the right people, tools and priority are applied from the start.

Question 13: Reusing Historical Defect Data for Regression Testing

Question: A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer

Regression testing confirms that changes have not broken existing functionality. Historical defect data shows which areas have been fragile in the past, so it can be used to build a regression suite that is both focused and effective.

Key Points

- Add a test case for every previously fixed defect to verify it does not reappear.
- Identify modules with high defect density and give them priority in the regression suite.
- Select regression tests according to the modules impacted by the current changes.
- Automate stable, frequently executed tests to save time on repeated runs.
- Review the suite after each release and remove outdated tests or add new ones based on fresh defect data.

Example

If a tax calculation defect recurred in three earlier releases, the team places tax-calculation tests at the top of the regression suite and runs them on every build.

Conclusion

Using defect history increases coverage of risky areas while avoiding unnecessary tests, which improves both coverage and efficiency.

Question 14: Tester's Role in Reducing Integration Defects

Question: During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer

Many integration defects come from misunderstandings at the interfaces between modules and between teams. A tester reduces them by acting as a communication link and quality advocate, from the start of the project.

Key Points

- Review interface specifications and data contracts for mismatches in format, units and protocols.
- Attend design discussions, stand-ups and review meetings alongside developers.
- Write clear, reproducible defect reports with steps, expected and actual results and environment details.
- Share test scenarios and acceptance criteria with developers early.
- Plan integration test environments and test data in advance.

Example

During design review, a tester notes that one module sends dates as DD/MM/YYYY while another expects YYYY-MM-DD. The mismatch is fixed on paper, avoiding several integration failures later.

Conclusion

Early involvement makes expectations common to all parties, reduces rework and builds a collaborative culture focused on quality.

Question 15: Defect Prevention Techniques

Question: *A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.*

Answer

When similar defects appear in every release, finding and fixing them each time is not enough. The team needs defect prevention, which identifies the root causes and changes the process so that the defects are not introduced at all.

Key Points

- Root cause analysis (RCA) of recurring defects, for example using a cause-and-effect diagram or the 5 Whys.
- Coding standards, design checklists and peer reviews to remove common mistakes.
- Static code analysis tools integrated into the build process.
- Training for developers and testers in the areas where errors recur.
- Lessons-learned sessions and regular feedback from the defect repository into the process.

Example

If null-pointer defects recur, RCA may show missing input checks. The team then adds a null-handling coding standard, enables a static-analysis rule that flags unchecked values and adds a review checklist item.

Conclusion

Prevention is cheaper than detection. It reduces rework, stabilises releases and raises overall software quality.

Question 16: Structured Testing Strategy for a Mobile Application

Question: *A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.*

Answer

Frequent crashes caused by inconsistent testing show the need for a standard strategy. Applying testing principles means planning what to test, how to test it and when to stop, instead of relying on individual preference.

Key Points

- Define test objectives, scope and exit criteria based on risk.
- Build a device, OS-version and screen-size matrix from user data.
- Cover functional, usability, performance, compatibility, interruption (calls, notifications) and network-variation testing.

- Use design techniques (equivalence partitioning, BVA, state transition) and automate repetitive checks.
- Analyse crash logs, record defects in a tracker and run regression on every build.

Example

Testing the app on low-memory devices with poor network conditions and with incoming calls during checkout shows crash scenarios that earlier ad hoc testing never exercised.

Conclusion

A structured approach makes results repeatable and coverage broader, which improves reliability and reduces crashes in the field.

Question 17: Testing versus Debugging

Question: *A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.*

Answer

Testing and debugging are related but separate activities and are normally performed by different roles. Confusing them delays defect resolution, because the team does not know who should do what and when.

Aspect	Testing	Debugging
Purpose	To detect failures and show that software meets requirements	To locate the cause of a failure and remove it
Performed by	Tester (also developers at unit level)	Developer
Outcome	Defect report with evidence	Corrected code
Nature	Planned and can be scheduled	Reactive; follows a reported defect
Follow-up	Retesting and regression testing	Verification by the tester

Key Points

- Testing finds that something is wrong; debugging finds why it is wrong and corrects it.
- The two activities form a cycle: test, report, debug, retest.

Example

On a shopping website, a tester finds that the order total ignores a coupon and reports it with steps. The developer debugs the discount function, finds a wrong condition and fixes it. The tester then retests and runs regression tests.

Conclusion

Testing provides the information that makes debugging possible, and debugging enables the verified fixes that improve quality. Both are needed, with clear roles.

Question 18: Cross-Browser Testing Strategy

Question: *An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.*

Answer

When an online shopping system behaves differently on different browsers, a cross-browser (compatibility) testing strategy is needed. It makes sure that the application gives a consistent experience on all supported browsers, versions and devices.

Key Points

- Select target browsers, versions and operating systems using real user analytics.
- Create a compatibility matrix and prioritise the combinations with the most users.
- Test layout, fonts, forms, JavaScript behaviour, cookies, media and responsive design on each combination.
- Combine manual exploratory checks with automated tools such as Selenium or cloud services like BrowserStack.
- Log defects with the browser and version, and include cross-browser checks in every regression cycle.

Example

The checkout button works in Chrome but is hidden in Safari because of a CSS incompatibility. The matrix-based test cycle finds this before release, and the CSS is corrected to follow standards.

Conclusion

Following web standards, testing on a defined matrix and automating regression checks effectively handle browser differences and protect customer satisfaction.

Question 19: Identifying Validation Gaps in a Logistics Application

Question: *A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.*

Answer

Inconsistent outputs usually result from incomplete input validation. Testing concepts help to identify the gaps systematically by examining what the system accepts, rejects and silently transforms.

Key Points

- Review the requirements for validation rules on every field: type, length, range, format and mandatory status.
- Use equivalence partitioning and boundary value analysis to test valid and invalid ranges.
- Add negative tests: empty values, negative weights, invalid PIN codes, wrong date formats and special characters.
- Check validation on both client and server side.
- Include data-flow checks to confirm validated values are not altered between modules.

Example

A shipment form accepts a negative package weight and produces a negative freight charge. A negative test on the weight field would have shown this validation gap immediately.

Conclusion

Proper validation testing prevents bad data from entering the system, and this makes outputs consistent and the system more reliable.

Question 20: Systematic Test Design as an Engineering Activity

Question: A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

Answer

Skipping test design and running random tests leaves coverage to chance. Applying testing as an engineering activity means each test is deliberately designed from the requirements before it is executed.

Key Points

- Analyse requirements and identify test conditions.
- Select suitable techniques (equivalence partitioning, BVA, decision tables, state transition).
- Write test cases with ID, preconditions, steps, test data and expected result.
- Review the test cases and map them to requirements through traceability.
- Execute, record actual results and report defects with evidence.

Sample Test Cases

TC ID	Input	Expected Result
TC-01	Valid user name and password	Login successful
TC-02	Valid user name, wrong password	Error message; access denied
TC-03	Empty user name and password	Mandatory-field message
TC-04	Password of maximum length + 1	Validation message shown

Example

The sample test cases above show how a login form is covered systematically: valid, invalid, empty and boundary inputs, each with a predefined expected outcome.

Conclusion

Systematic design is important because it gives full coverage, repeatability, reusability and traceability, and it detects defects that random testing would miss.

Question 21: Data Inconsistency across Healthcare Modules

Question: A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

Answer

When modules store or exchange the same data differently, inconsistencies appear. Testing principles guide the team to trace the data through the system and find where it changes or is lost.

Key Points

- Likely root causes: mismatched data types, formats or units at interfaces; wrong field mapping; missing transaction handling; duplicate data stores; concurrency issues.
- Apply integration and interface testing to verify data passed between modules.
- Use data-flow and end-to-end tests that follow a record from entry to final use.
- Reconcile database values after each operation (create, update, delete) to confirm consistency.
- Test concurrent access and failure recovery to ensure updates are complete or fully rolled back.

Example

A patient's address is updated in the registration module but the billing module still shows the old address. An end-to-end test that updates a record and then checks every dependent module would have exposed the missing synchronisation.

Conclusion

Thorough integration and data-flow testing ensures that all modules hold the same, correct data, which is essential in healthcare.

Question 22: Role of Testers across SDLC Phases

Question: A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

Answer

Testers add value in every phase of the SDLC, not only during test execution. Involving them from the beginning prevents the delays that occur when problems are discovered late.

SDLC Phase	Role of the Tester
Requirements	Review requirements for clarity and testability; define acceptance criteria
Design	Review design; plan test strategy, test environment and data
Coding	Prepare test cases; support unit testing and code reviews
Testing	Execute tests, report defects, perform retesting and regression
Deployment	Perform smoke and acceptance testing; verify release
Maintenance	Run regression tests for changes and analyse production defects

Key Points

- Testers who know the system from the start can begin preparation earlier, so the test phase is shorter.
- Defects found in early phases need less rework and avoid schedule slips.
- Regular communication with developers and analysts removes misunderstandings.

Example

In a project where the tester joins at the design stage, a missing error-handling flow is found in a design review, saving several days of rework during system testing.

Conclusion

Early and continuous involvement of testers improves quality, shortens delivery time and improves collaboration.

Question 23: Test Coverage and Peak-Load Failures

Question: *A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.*

Answer

A failure at peak usage shows that important scenarios were never tested. The gap is typically a lack of non-functional testing (load, stress, spike) and of realistic usage scenarios.

Key Points

- Identify gaps by comparing the test suite with requirements, the risk list and expected usage profiles.
- Check coverage measures: requirement coverage, scenario coverage and code coverage.
- Add load, stress, spike and endurance tests that simulate peak concurrent users.
- Include negative and rare scenarios such as timeouts, failed payments and session expiry.
- Use risk-based testing to make sure high-impact scenarios are tested first.

Example

An online ticketing system works with 100 users but fails when 10,000 users book at the same time during a sale. A planned spike test would have shown this bottleneck in advance.

Conclusion

Better coverage, especially of realistic peak scenarios, reveals weaknesses before release and prevents failures when demand is highest.

Question 24: Defect Repository and Documentation

Question: A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

Answer

Without proper defect documentation, defects are lost, duplicated or misunderstood. A defect repository solves this by recording every defect in a standard format and tracking it through its life cycle.

Key Points

- Each record contains: defect ID, title, description, steps to reproduce, expected and actual result, environment, module, severity, priority, status, owner and attachments.
- The defect life cycle is tracked: New, Assigned, Open, Fixed, Retest, Verified, Closed (with Reopened, Rejected or Deferred as alternatives).
- A tool such as Jira or Bugzilla gives a single, shared source of truth.
- Search and reporting functions enable analysis of trends, aging and defects per module.
- Documented history supports root cause analysis and process improvement.

Example

A report that states 'login not working' is hard to act on. A documented report with steps, browser, expected and actual result lets a developer reproduce and fix the defect quickly.

Conclusion

Good documentation makes tracking transparent, speeds up resolution and provides data for analysis, which improves defect management.

Question 25: Redesigning the Testing Approach for a Banking System

Question: A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

Answer

Repeated transaction failures indicate that testing is not matched to the risks of the system. The approach must be redesigned around the critical nature of banking transactions.

Key Points

- Apply risk-based test planning, with priority for transfers, deposits, withdrawals and interest.
- Verify transaction integrity (atomicity, consistency, isolation, durability) including rollback after failure.
- Use decision tables and state transition testing for transaction rules and states.
- Include concurrency, security and performance tests for simultaneous transactions.
- Maintain an automated regression suite and use the defect repository to refine tests each cycle.

Example

Two simultaneous withdrawals from the same account are tested to confirm that the balance never becomes negative. Such a concurrency test finds failures that single-user tests cannot.

Conclusion

Structured, risk-based testing gives complete and repeatable coverage of transaction logic, improving the reliability of the banking system.

Question 26: Edge Case Testing and Boundary Testing

Question: *A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.*

Answer

Testing only normal inputs leaves the extremes of the input domain unchecked, and these are where failures are most likely. Testing principles require that valid, invalid and limit values are all included.

Key Points

- Identify the input ranges and limits from the requirements.

- Apply equivalence partitioning to group inputs, then boundary value analysis for the edges of each group.
- Include edge values: zero, minimum, maximum, just inside and just outside the range, empty, null and very long inputs.
- Add error guessing based on past defects.
- Document each case with the expected outcome.

Example

A transfer limit of 1 to 50,000 requires tests with 0, 1, 2, 49,999, 50,000 and 50,001. The values at the edges often expose off-by-one logic errors.

Conclusion

Boundary and edge-case testing detects defects that normal-input testing misses, so the number of detected defects rises sharply for a small additional effort.

Question 27: Process-Based Testing and Standardisation

Question: *A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.*

Answer

Uneven quality across modules arises when each team follows its own method. A process-based approach defines one common way of working that all teams follow.

Key Points

- Adopt a standard test process with defined phases: plan, design, execute, report and close.
- Use common templates for test plans, test cases and defect reports.
- Define uniform entry/exit criteria, review checklists and naming conventions.
- Use one test-management tool and a shared set of metrics.
- Train the teams and audit compliance; improve the process with frameworks such as TMMi or ISO/IEC/IEEE 29119.

Example

When every module uses the same defect report template and exit criteria of 'no open high-severity defects', quality can be compared fairly across modules and weak areas become visible.

Conclusion

Standardisation brings consistent coverage, comparable results and easier management, so overall quality becomes even across modules.

Question 28: Input Validation and Robustness Testing

Question: *A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.*

Answer

A web application that fails on unexpected input lacks robust validation. Testing must deliberately supply invalid, unusual and malicious-looking data to check that the system handles it safely.

Key Points

- Test wrong data types, empty fields, excessive length, special characters, Unicode and script-like strings.
- Apply negative testing and error guessing in addition to normal positive tests.
- Confirm validation on both client and server side, since client checks can be bypassed.
- Verify that the system shows clear, safe error messages and does not crash or expose internal details.
- Add automated input tests (including fuzzing) to the regression suite.

Example

A search box receives a 10,000-character string and special symbols. A robust application rejects or sanitises the input with a friendly message, while a weak one throws an unhandled error.

Conclusion

Proper validation testing makes the application robust: it continues to behave correctly and securely when users supply unexpected data.

Question 29: Defect Leakage: Root Cause and Prevention

Question: A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

Answer

Defect leakage means defects that escape testing and are found in production. It is measured as:
$$\text{Defect Leakage (\%)} = (\text{defects found after release} / \text{total defects found}) \times 100.$$
A high value shows that testing is not effective enough.

Key Points

- Possible root causes: incomplete requirement coverage, weak test design, insufficient regression, test environment differing from production, poor test data, schedule pressure and missed non-functional testing.
- Analyse leaked defects to find in which phase they should have been caught.
- Improve test design and coverage for the identified gaps.
- Make the test environment and data close to production.
- Strengthen regression, reviews and exit criteria before release.

Example

If most leaked defects are found only on real mobile networks, adding network-condition tests and a production-like environment prevents similar leaks.

Conclusion

Root cause analysis combined with better coverage, environments and release criteria reduces leakage and customer-reported defects.

Question 30: Early Defect Detection Strategy

Question: *A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.*

Answer

When defects are found late, cost and effort rise. The strategy should be redesigned around the 'shift-left' approach, which moves testing activities as early as possible.

Key Points

- Use static testing: reviews, walkthroughs and inspections of requirements, design and code.
- Plan tests and write test cases during requirement and design phases.
- Encourage unit testing and static analysis by developers.
- Use continuous integration with automated tests so that problems are noticed within hours.
- Apply risk-based prioritisation to test the riskiest parts first.

Example

A design review reveals a missing exception path. Fixing it takes one hour in the design document, whereas the same defect found in production would require code change, retesting, redeployment and customer support.

Conclusion

Early defect detection reduces rework, shortens schedules and lowers cost, which makes the whole project more efficient.